Centralized Real-Time Queue Management Platform

Architecture Specification — `architecture.md`

---

1. Document Information

Field	Details
Project	Centralized Real-Time Queue Management Platform
Document	architecture.md
Version	1.0
Status	Proposed
Frontend	Next.js

| Backend | Node.js |

| Database | PostgreSQL |

| Architecture | Multi-Tenant Web Application |

| Real-Time | Real-Time Queue State Synchronization |

| Notifications | WhatsApp + SMS |

---

2. Architecture Objective

The system will provide a centralized, real-time queue management platform that can be used across different industries.

The platform must support:

- Multiple tenants/organizations.

- Independent queues for each tenant.

- Admin-generated QR codes.

- QR-based queue joining.

- Automatic queue numbering.

- First-Come-First-Served queue behavior.

- Admin-controlled queue reordering.

- Real-time queue position updates.

- Queue completion using DONE.

- User cancellation.

- Queue-clearing notifications.

- Approximate waiting-time calculation.

- WhatsApp notifications.

- SMS notifications.

- Secure tenant isolation.

- Responsive mobile and desktop interfaces.

The architecture must remain generic and must not depend on a specific industry.

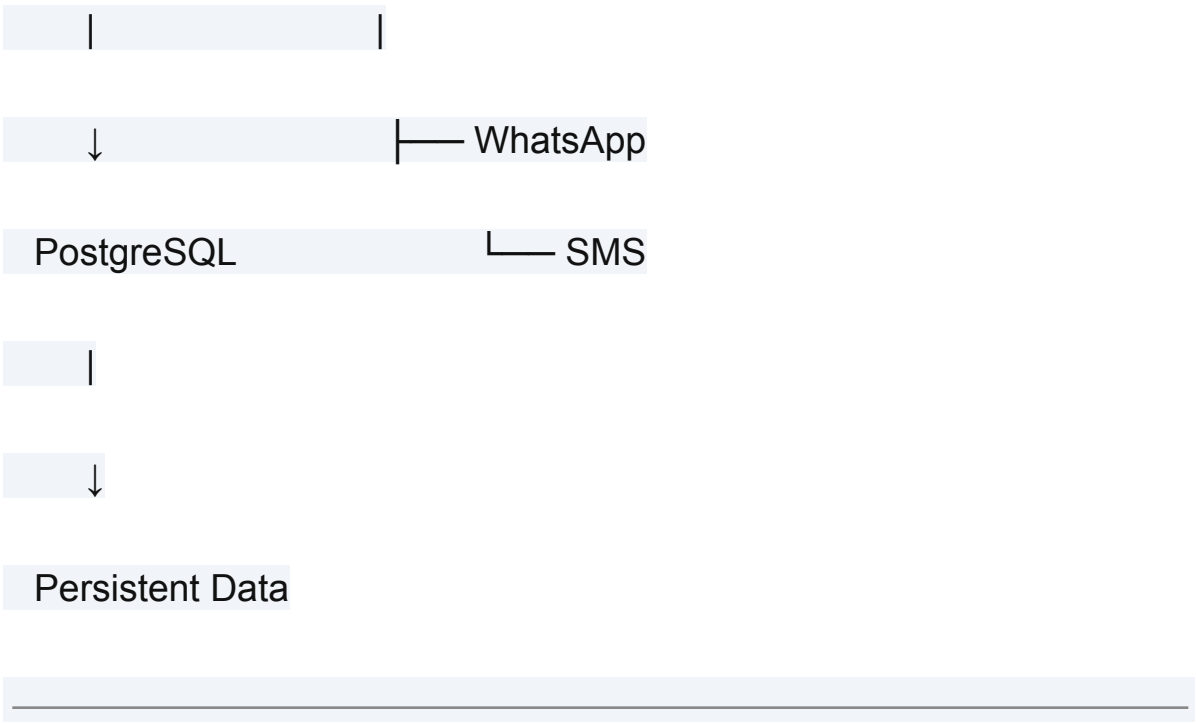
---

3. High-Level Architecture

The proposed architecture is:

```text
                    USERS
                      |
          ┌─────────────────────────┐
```


End Users
Admin / Operator
Next.js
Frontend
Node.js API
Queue Logic
Auth/Security
Notification



---

4. Main Architectural Components

The application consists of the following logical components:

1. Next.js Web Application

2. Node.js Backend API

3. Queue Management Module

4. Authentication & Authorization Module

5. Multi-Tenant Access Layer

6. PostgreSQL Database

7. Real-Time Communication Layer

8. Notification Service

9. WhatsApp Integration

10. SMS Integration

11. Validation Layer

12. Rate Limiting Layer

13. Audit/Event Layer

These are logical components and do not necessarily require separate deployable services.

The initial architecture should remain as simple as possible.

---

5. Frontend Architecture

Next.js will provide the web application.

The frontend should be divided conceptually into:

Next.js

|

├── Public Queue Experience

│

├── User Queue Status

│

├── Operator Dashboard

│

├── Organization Admin Dashboard

│

└── Platform Admin Interface

---

6. End-User Application

The end-user flow is designed primarily for mobile devices.

User

↓

Scan QR

↓

Open Queue Page

↓

Enter Details

↓

Join Queue

↓

Receive Queue Position

↓

View Live Status

The interface should show information such as:

Queue Name

Your Position

#4

People Ahead

3

Estimated Wait

Approximately 25 minutes

[Leave Queue]

The UI must update when the backend queue state changes.

---

7. Admin Application

The Organization Admin interface should provide:

- Queue creation.
- Queue configuration.
- QR generation.
- Queue monitoring.
- Queue position modification.
- Authorized operator management where required.
- Queue status management.

- Tenant-specific information.

Example:

```
Admin Dashboard
      |
      ├── Create Queue
      ├── Generate QR
      ├── View Queue
      ├── Modify Position
      └── Queue Settings
```

---

8. Operator Application

The operator interface is responsible for active queue operations.

Example:

```
Operator Dashboard



Current Queue
```

#1 Person A [DONE]

#2 Person B

#3 Person C

#4 Person D

When the operator clicks `DONE`:

Person A

↓

COMPLETED

↓

Removed from Active Queue

↓

Queue Recalculated

---

9. Backend Architecture

Node.js will contain the authoritative application logic.

Conceptually:

HTTP Request

↓

Authentication

↓

Authorization

↓

Tenant Verification

↓

Request Validation

↓

Business Logic

↓

Database Transaction

↓

Response

The backend must not trust the frontend.

---

10. Backend Modules

The backend should be organized into logical modules.

Backend

│

├── Authentication

├── Authorization

├── Tenancy

├── Queues

├── Queue Entries

├── Queue Events

├── Notifications

```
├── QR Management
├── Waiting-Time Calculation
├── Audit
└── Shared Infrastructure
```

---

11. Multi-Tenant Architecture

Multi-tenancy is a mandatory requirement.

The platform supports:

```
Central Platform
    |
    ├── Tenant A
    |     ├── Queue A1
    |     └── Queue A2
    |
    ├── Tenant B
```

```
    |   ├── Queue B1
    |   └── Queue B2
    |
    └── Tenant C
        └── Queue C1
```

Every tenant operates independently.

---

12. Tenant Resolution

For authenticated users, tenant context should be established from their authenticated identity and permissions.

For public queue access through QR, the QR identifier/token should identify the intended queue.

Conceptually:

```
Authenticated Request
    ↓
Authenticated User
    ↓
```

Tenant Context

↓

Authorization

↓

Tenant Data

Public queue access:

QR

↓

Queue Identifier / Secure Token

↓

Validate Queue

↓

Validate Tenant Relationship

↓

Public Queue Flow

---

13. Tenant Isolation

Tenant isolation must be enforced at the backend.

Every tenant-owned database operation must ensure that the requested data belongs to the current tenant.

Example:

```
Tenant A Request

    |

    ↓

Query Tenant A Data

    |

    X

Tenant B Data
```

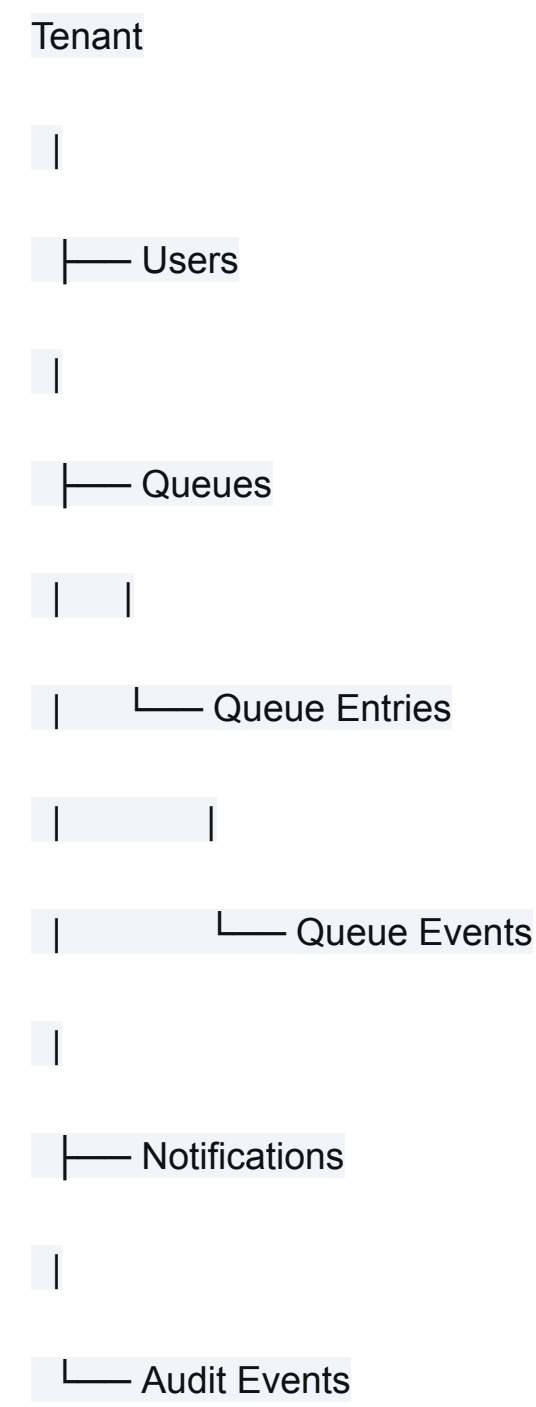
The frontend must never be responsible for enforcing this boundary.

---

14. Database Architecture

PostgreSQL is the persistent source of truth.

Core relationships:



The detailed database model is defined in `database.md`.

---

15. Queue Lifecycle

The standard queue lifecycle is:

Queue Created

↓

QR Generated

↓

Queue Open

↓

Users Join

↓

Users Wait

↓

Users Called/Served

↓

DONE

↓

Completed

↓

Next Users Move Forward

---

16. User Queue Entry Lifecycle

A queue entry can follow:

WAITING

|

↓

CALLED

|

↓

COMPLETED

Alternative terminal states may include:

CANCELLED

EXPIRED

NO_SHOW

The backend must enforce valid state transitions.

---

17. Queue Joining Flow

The standard user joining process is:

User

↓

Scan QR

↓

Queue Registration Page

↓

Enter Required Information

↓

Submit

↓

Backend Validation

↓

Check Queue Availability

↓

Create Queue Entry

↓

Assign Queue Ordering

↓

Persist Entry

↓

Return Queue Status

---

18. Automatic Queue Numbering

The default behavior is First-Come-First-Served.

Example:

Person 1 → #1

Person 2 → #2

Person 3 → #3

Person 4 → #4

When users join simultaneously, the backend/database must ensure that each entry receives a unique and deterministic ordering.

---

19. Queue Position Model

The system should distinguish:

Queue Entry Identity

+

Queue Ordering

↓

Current Position

The current displayed position should be derived from the active queue ordering.

Example:

Before:

A → #1

B → #2

C → #3

D → #4

After A is completed:

B → #1

C → #2

D → #3

The historical identity of the queue entries remains intact.

---

20. DONE Flow

When an operator clicks `DONE`:

Operator

↓

DONE

↓

Backend validates action

↓

Database transaction

↓

Entry marked COMPLETED

↓

Completion time recorded

↓

Queue event recorded

↓

Active queue changes

↓

Real-time update published

↓

Users see updated positions

The completed entry must remain available for historical/analytics purposes where permitted.

---

21. Admin Reordering Flow

An Organization Admin can manually change a user's position.

Example:

Before:

#1 A

#2 B

#3 C

#4 D

Admin moves D to #2:

After:

#1 A

#2 D

#3 B

#4 C

The backend must:

1. Authenticate the admin.
2. Verify organization access.
3. Verify queue access.
4. Validate the requested position.
5. Apply the change transactionally.
6. Record the event.
7. Publish the new queue state.

---

22. VIP/Priority Behavior

No separate VIP queue is required for the core system.

Instead:

Default

↓

First-Come-First-Served

Admin Override

↓

Manual Position Change

This allows an administrator to prioritize a user when necessary.

---

23. User Cancellation Flow

A user can leave the queue.

Frontend:

[Leave Queue]

↓

Confirmation

↓

"You will lose your current position."

↓

Confirm

Backend:

Validate Entry

↓

Update Status

↓

Record Cancellation

↓

Remove from Active Queue

↓

Update Queue

↓

Publish Real-Time Change

---

24. Queue Clearing Flow

If multiple users are completed or removed:

Queue State Changes

↓

Backend Updates Database

↓

Active Queue Recalculated

↓

Real-Time State Published

↓

Affected Users Updated

↓

Queue-Cleared Notification if applicable

If the queue becomes empty:

QUEUE CLEARED

The appropriate notification must be generated.

---

25. Real-Time Architecture

Real-time updates are required so that users do not need to manually refresh the page.

Conceptually:

Operator Action

↓

Node.js Backend

↓

PostgreSQL

↓

Queue State Changed

↓

Real-Time Event

↓

Connected Clients

↓

Updated UI

A WebSocket-based approach is a strong candidate.

The exact implementation/library will be selected during technical implementation.

---

26. Real-Time Source of Truth

The real-time layer is not the source of truth.

PostgreSQL

↓

Source of Truth

Real-Time Layer

↓

State Change Delivery

If a client disconnects, it can obtain the latest state from the backend/database after reconnecting.

---

27. Real-Time Event Types

Potential events include:

QUEUE_UPDATED

QUEUE_POSITION_CHANGED

QUEUE_ENTRY_JOINED

QUEUE_ENTRY_COMPLETED

QUEUE_ENTRY_CANCELLED

QUEUE_CALLED

QUEUE_REORDERED

QUEUE_CLEARED

QUEUE_PAUSED

QUEUE_CLOSED

The final event list may be adjusted during implementation.

---

28. Real-Time Connection Recovery

If a user loses connectivity:

Connection Lost

↓

Reconnect

↓

Request Latest Queue State

↓

Synchronize

↓

Resume Real-Time Updates

The system must not rely on the client receiving every event to maintain correctness.

The latest persisted backend state always takes precedence.

---

29. Estimated Waiting-Time Architecture

The waiting-time system uses actual historical service durations.

Completed Queue Entries

↓

Actual Service Durations

↓

Average Service Time

↓

People Ahead

↓

Estimated Waiting Time

Formula:

Estimated Wait

=

People Ahead × Average Service Time

Example:

Previous service durations:

8 min

10 min

7 min

9 min

Average = 8.5 min

Current position = #4

People ahead = 3

Estimated Wait

≈ 3 × 8.5

≈ 25.5 minutes

The estimate should be presented as approximate.

---

30. Waiting-Time Updates

The estimated waiting time should be recalculated when relevant queue state changes.

Potential triggers:

- New user joins.
- User ahead completes.
- User ahead cancels.
- Admin reorders queue.
- New service-duration information becomes available.

Example:

Position #4

Estimated Wait: 25 min

Person ahead completes

Position #3

Estimated Wait: 17 min

---

31. Insufficient Historical Data

If insufficient service-time history exists, the system must avoid presenting false precision.

The architecture should support a fallback such as:

Calculating estimated wait...

or another configured fallback strategy.

The exact fallback value/behavior must be defined before production.

---

32. Notification Architecture

Notifications are handled separately from core queue processing.

```
Queue Event
    ↓
Notification Service
    |
    ├── WhatsApp
    |
    └── SMS
```

Both WhatsApp and SMS are first-class notification channels.

SMS is not assumed to be only a fallback.

---

33. Notification Processing

The recommended logical flow is:

```
Queue Event
```

↓

Database Transaction

↓

Commit

↓

Notification Event

↓

Notification Service

↓

WhatsApp / SMS

Notification failure should not undo a successful queue operation.

---

34. Notification Events

Important events may trigger external notifications, such as:

- Queue joined.
- User approaching turn.
- Turn available.

- Queue suddenly clears.
- Queue cancelled/closed.
- Other important business events.

Real-time position changes can be displayed directly in the web application without necessarily sending an external message for every single position change.

---

35. Notification Configuration

Notification functionality must be configurable through environment/configuration.

Conceptually:

```
WHATSAPP_ENABLED

SMS_ENABLED

WHATSAPP_PROVIDER

SMS_PROVIDER
```

Provider credentials must be stored securely.

Specific providers are not locked by this architecture document.

---

36. Notification Delivery Tracking

Notification delivery should be tracked.

Example:

Notification

↓

WhatsApp

↓

SENT

↓

DELIVERED / FAILED

And independently:

Notification

↓

SMS

↓

SENT

↓

DELIVERED / FAILED

This allows the system to track delivery problems without affecting queue state.

---

37. QR Architecture

Only an Organization Admin can generate QR codes.

Flow:

Organization Admin

↓

Create Queue

↓

Generate QR

↓

QR linked to Queue

↓

Display/Print QR

↓

User Scans QR

↓

Public Queue Entry

---

38. QR Security

The QR must identify the intended queue without exposing sensitive information.

A secure, non-guessable queue identifier/token should be considered for public QR URLs.

When scanned:

QR Token

↓

Backend

↓

Validate Token

↓

Find Queue

↓

Check Queue Status

↓

Allow/Reject Join

Users must not be able to modify the QR's associated queue.

---

39. Authentication Architecture

Authentication is required for privileged users.

Admin / Operator

↓

Login

↓

Authentication

↓

Session / Token

↓

Authorized API Access

End users may use public queue functionality without a permanent account.

---

40. Authorization Architecture

Authorization is enforced by the backend.

Request

↓

Authenticated User

↓

Role Check

↓

Tenant Check

↓

Resource/Queue Permission

↓

Allow / Reject

Example:

Organization Admin

→ Manage tenant queues

Queue Operator

→ Operate authorized queues

End User

→ Access own queue session/status

---

41. Public User Security

Public queue access must be carefully restricted.

A user should only be able to access the queue information associated with their own secure queue session/status link where required.

The API must prevent:

Entry ID 100

↓

Change URL/Request

↓

Access Entry 101

Public identifiers should not be easily enumerable.

---

42. API Architecture

The Node.js backend will expose APIs for:

Authentication

Tenants

- Users
- Queues
- Queue Entries
- Queue Events
- QR Codes
- Notifications
- Admin Operations
- Operator Operations

Example logical API groups:

- /api/auth/*
- /api/tenants/*
- /api/queues/*
- /api/queue-entries/*
- /api/notifications/*
- /api/admin/*

The exact endpoint naming will be finalized during implementation.

---

43. API Request Flow

Every protected API should follow:

Request

↓

Rate Limit

↓

Authentication

↓

Authorization

↓

Tenant Verification

↓

Schema Validation

↓

Business Logic

↓

Database

↓

Response

---

44. Validation Architecture

Validation must happen at multiple levels.

User Input

↓

Frontend Validation

↓

API Request

↓

Backend Schema Validation

↓

Business Validation

↓

Database Constraints

Frontend validation improves user experience.

Backend validation provides security and correctness.

Database constraints provide an additional integrity layer.

---

45. Rate Limiting Architecture

Rate limiting must protect public and sensitive endpoints.

Potential targets:

Authentication

Queue Joining

Public Queue Status

QR/Public Endpoints

Admin APIs

Notification APIs

The exact numerical limits will be configured according to endpoint sensitivity.

Example configurable limits may include:

5 requests / time window

10 requests / time window

The exact production values will be finalized during security implementation.

---

46. Error Handling Architecture

Errors should be handled consistently.

Backend Error

↓

Internal Logging

↓

Safe API Error Response

↓

Frontend Error State

↓

User-Friendly Message

Internal stack traces, credentials, database details, and sensitive information must not be exposed to users.

---

47. Database Transaction Architecture

Critical queue operations must use database transactions where necessary.

Join

Validate

↓

Create Entry

↓

Assign Ordering

↓

Commit

DONE

Validate

↓

Complete Entry

↓

Record Event

↓

Commit

↓

Publish Update

Reorder

Validate

↓

Change Queue Order

↓

Record Event

↓

Commit

↓

Publish Update

---

48. Concurrency Architecture

The queue can receive simultaneous operations.

Example:

User A → JOIN

User B → JOIN

Operator → DONE

Admin → REORDER

The backend/database must maintain consistent queue state even when operations happen at the same time.

Concurrency controls and transactions must be used where necessary.

---

49. Notification Consistency

Queue state changes must be committed before external notifications are treated as valid.

Example:

Queue Operation

↓

Database Commit

↓

Notification Event

↓

WhatsApp / SMS

A failed WhatsApp/SMS delivery must not undo the database operation.

---

50. Event and Audit Architecture

Important actions should generate events.

Example:

Admin changes position

↓

Queue Event

↓

Audit Record

↓

Real-Time Update

This provides traceability and supports future analytics/debugging.

---

51. Security Architecture

Security must be implemented at multiple layers.

Browser

↓

API Security

↓

Authentication

↓

Authorization

↓

Tenant Isolation

↓

Validation

↓

Rate Limiting

↓

Database Security

Required security measures include:

- Authentication.
- Authorization.
- Tenant isolation.

- Input validation.
- Rate limiting.
- Secure sessions/tokens.
- Secure environment configuration.
- Database access controls.
- Audit logging.
- Secure error handling.

---

52. Environment Security

Secrets must never be committed to Git.

Examples:

Database Credentials

API Keys

WhatsApp Credentials

SMS Credentials

Authentication Secrets

Environment files containing secrets must be ignored.

Example:

.env

.env.*

!.env.example

---

53. Frontend Security Principle

The frontend must not be considered a trusted security boundary.

For example:

Frontend says:

"I am an Admin."

Backend:

"Verify it."

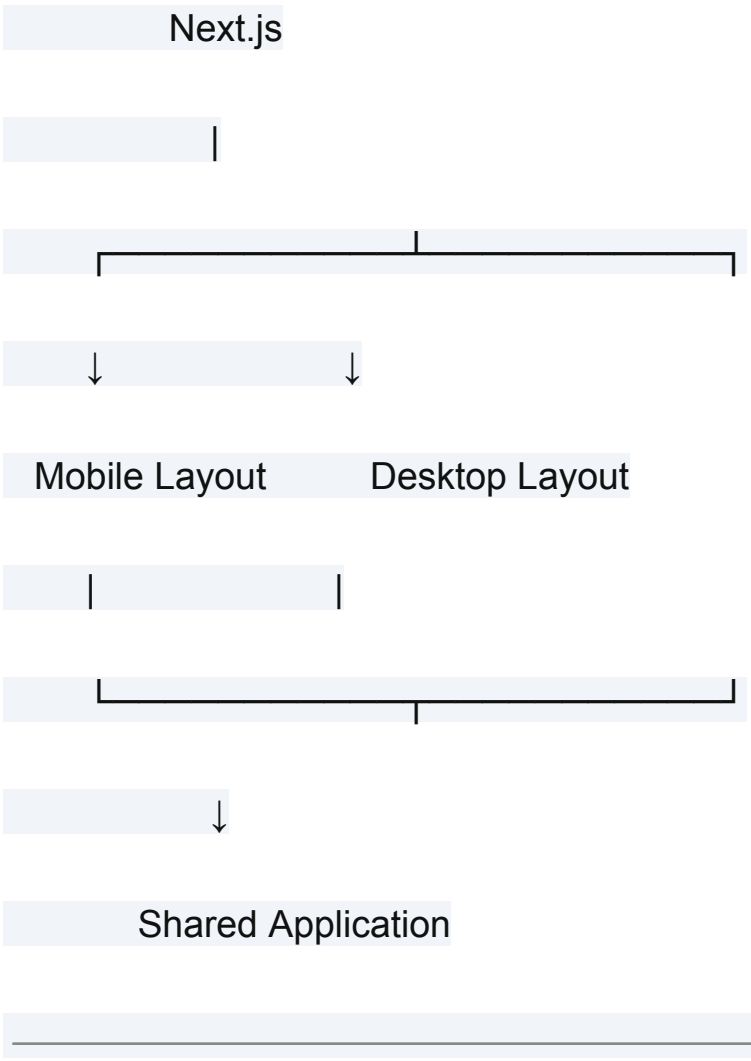
Every privileged action must be checked by the backend.

---

54. Responsive Architecture

Responsive behavior is part of the architecture and implementation process.

The frontend should use responsive layouts rather than separate desktop and mobile applications.



---

55. Mobile-First User Experience

The QR-based customer flow should be optimized for mobile.

Important principles:

- Large touch targets.
- Simple forms.

- Minimal steps.
- Clear queue position.
- Clear estimated wait.
- Easy cancellation.
- Clear notifications/status.
- No unnecessary navigation.

---

56. Desktop Admin Experience

The administrator/operator experience should provide efficient queue management.

The interface should support:

- Queue lists.
- Queue entries.
- Current positions.
- DONE actions.
- Admin position changes.
- Queue settings.
- QR generation.
- Status indicators.

The UI must remain responsive.

---

57. UI Architecture

Shadcn UI should be used for reusable components.

Conceptually:

```
Shared UI Components
        |
        ├── Buttons
        ├── Forms
        ├── Dialogs
        ├── Tables
        ├── Alerts
        ├── Status Indicators
        └── Navigation
```

Components should be reusable across the application.

---

58. Testing Architecture

Testing must exist at multiple levels.

Unit Tests

↓

Integration/API Tests

↓

End-to-End Tests

↓

Browser/Responsive Tests

Every important feature should contain:

Happy Flow

+

Negative Flow

---

59. Playwright Architecture

Playwright will be the primary E2E browser testing framework.

Important scenarios include:

Admin Login

↓

Create Queue

↓

Generate QR

↓

User Joins

↓

Second User Joins

↓

Queue Positions

↓

Operator DONE

↓

Positions Move Forward

↓

Real-Time Update

↓

User Turn

---

60. Selenium Architecture

Selenium can be used for additional browser compatibility/testing requirements.

It should not unnecessarily duplicate the complete Playwright test suite.

---

61. Husky Architecture

Husky will help enforce development quality before commits.

Example:

git commit

↓

Husky

↓

Lint

↓

Type Check

↓

Tests

↓

PASS → Commit

FAIL → Commit Blocked

The exact hooks will be configured during project setup.

---

62. Git Architecture

Development must use separate branches.

Example:

main

```
|
├── feature/queue-management
├── feature/notifications
├── feature/multitenancy
└── feature/admin
```

The `main` branch should remain stable.

Meaningful completed features should be committed at stable checkpoints.

---

63. Monorepo Architecture

A monorepo is a strong candidate because the system contains:

```
Next.js Frontend
      +
Node.js Backend
      +
Shared Types
```

```
      +
   Shared Validation
      +
   Shared UI
      +
   Tests
```

Possible structure:

```
project/
│
├── apps/
│   ├── web/
│   └── api/
│
├── packages/
│   ├── ui/
```

```
│   ├── types/
│   ├── validation/
│   └── config/
│
├── tests/
│
├── prd.md
├── techstack.md
├── database.md
├── architecture.md
└── development.md
```

The final monorepo tooling will be selected during scaffold design.

---

64. Service Boundaries

The initial implementation should avoid unnecessary microservices.

Recommended logical architecture:

```
Next.js
  ↓
Node.js Application
  |
  ├── Queue Module
  ├── Auth Module
  ├── Tenant Module
  ├── Notification Module
  ├── QR Module
  └── Audit/Event Module
  |
  ↓
PostgreSQL
```

The application can later be separated into independent services if scale or operational requirements justify it.

---

65. Deployment Architecture

The final deployment provider is not yet locked.

The architecture should support:

```
Internet
  ↓
Web Application
  ↓
Backend API
  ↓
PostgreSQL
```

External integrations:

```
Backend
  ├── WhatsApp Provider
  └── SMS Provider
```

The final deployment design will be documented during implementation

planning.

---

66. Logging

The backend should produce structured logs for important operations.

Examples:

- Queue Created
- Queue Entry Created
- Queue Entry Completed
- Queue Reordered
- Notification Sent
- Notification Failed
- Authentication Failure
- Authorization Failure
- Rate Limit Triggered

Logs must not contain sensitive credentials or unnecessary personal information.

---

67. Monitoring

Production monitoring should eventually cover:

- API availability.
- Database health.
- Queue operation errors.
- Real-time connection failures.
- Notification failures.
- Authentication failures.
- Rate-limit events.
- Application errors.

The exact monitoring platform will be selected during deployment planning.

---

68. Failure Handling

The system should fail safely.

Database Failure

Request

↓

Database unavailable

↓

Safe error response

↓

No incorrect queue state

Notification Failure

Queue operation succeeds

↓

WhatsApp/SMS fails

↓

Record delivery failure

↓

Queue remains correct

Real-Time Failure

Real-Time connection fails

↓

User reconnects

↓

Latest state fetched

↓

Correct queue displayed

---

69. Architecture Principles

The architecture must follow these principles:

Backend Authority

Important business logic belongs on the backend.

Database as Source of Truth

PostgreSQL stores authoritative persistent state.

Multi-Tenant Isolation

Tenant data must remain isolated.

Real-Time Without Data Loss

Real-time communication is for synchronization, not permanent state.

Secure by Default

Authentication, authorization, validation, and rate limiting are mandatory.

Simple First

Avoid unnecessary microservices and infrastructure.

Generic Design

Do not design the core system around one industry.

Responsive by Default

Mobile and desktop support must be built from the beginning.

Testable

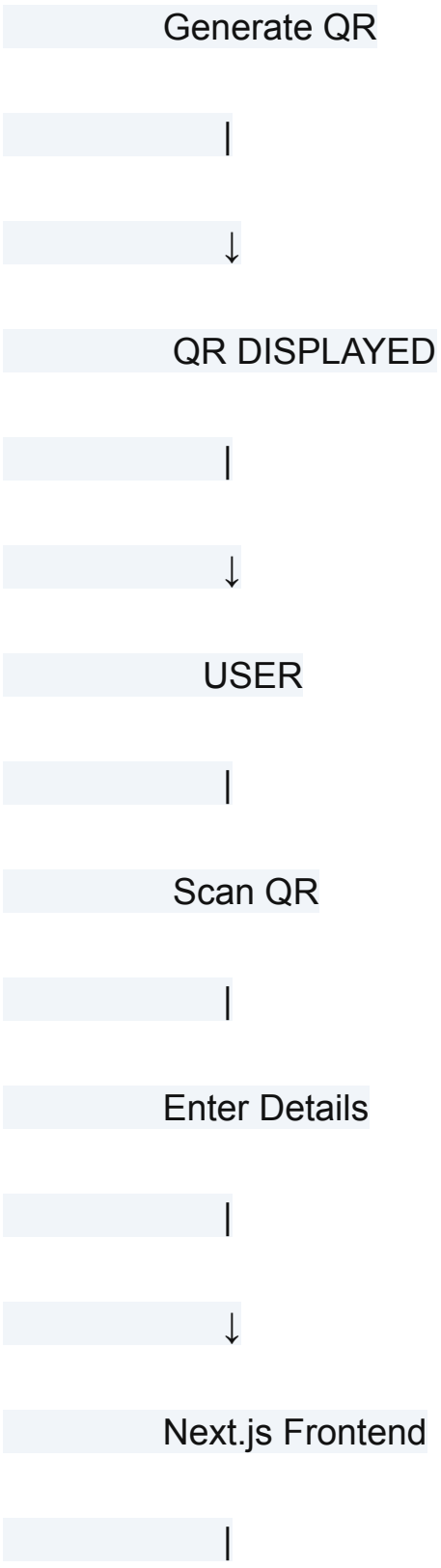
Every important feature must be testable.

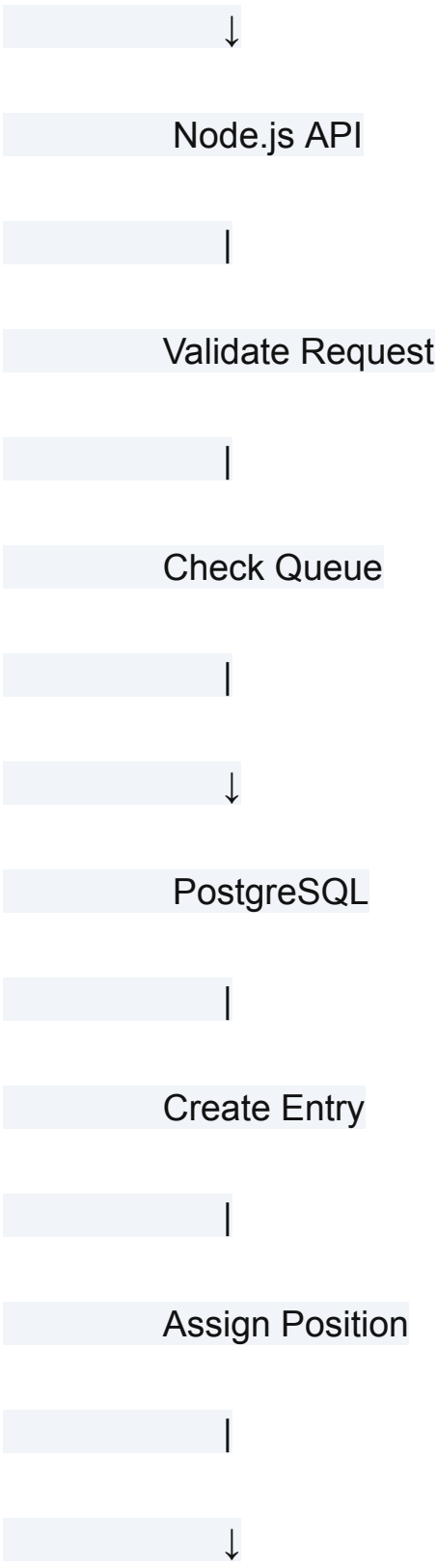
---

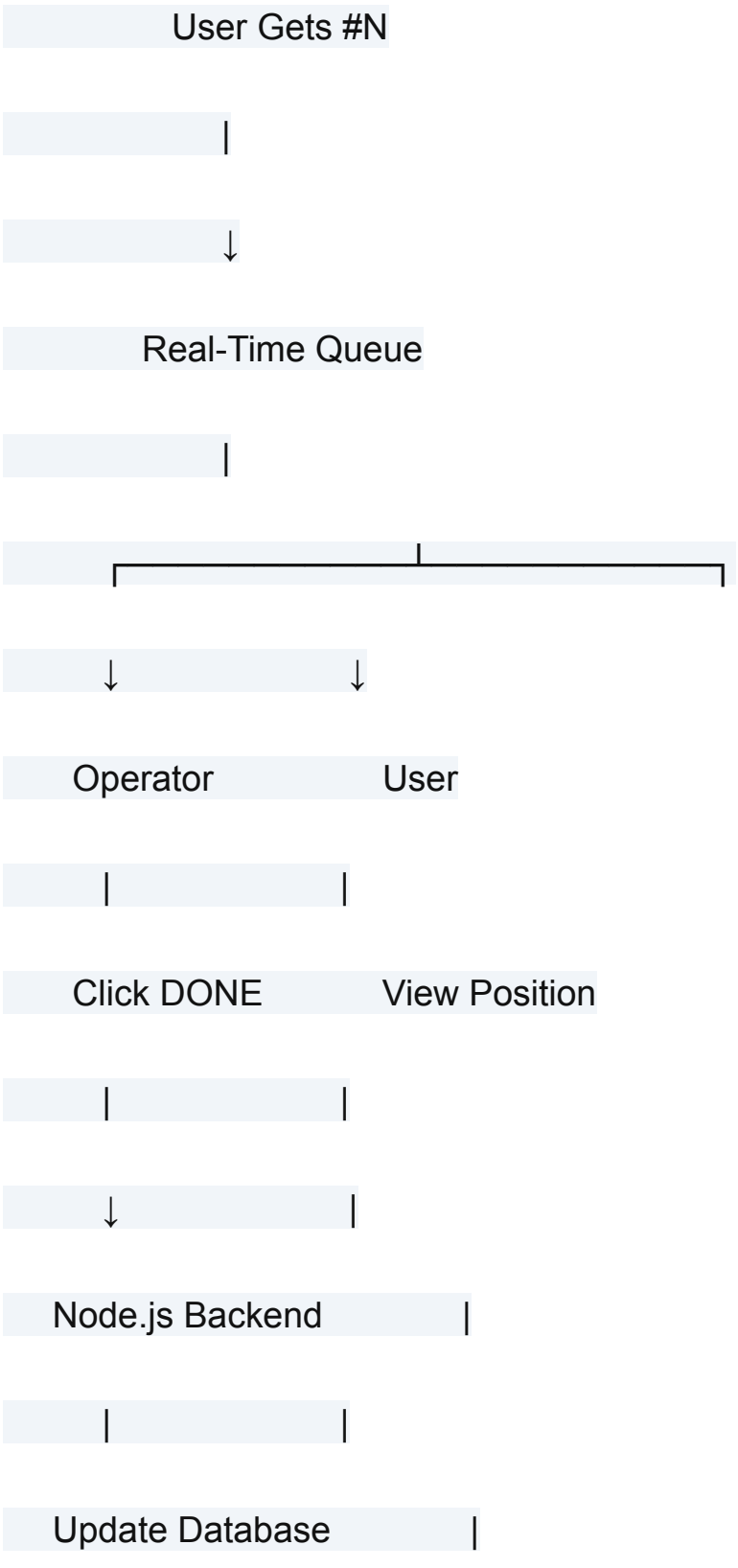
70. End-to-End System Flow

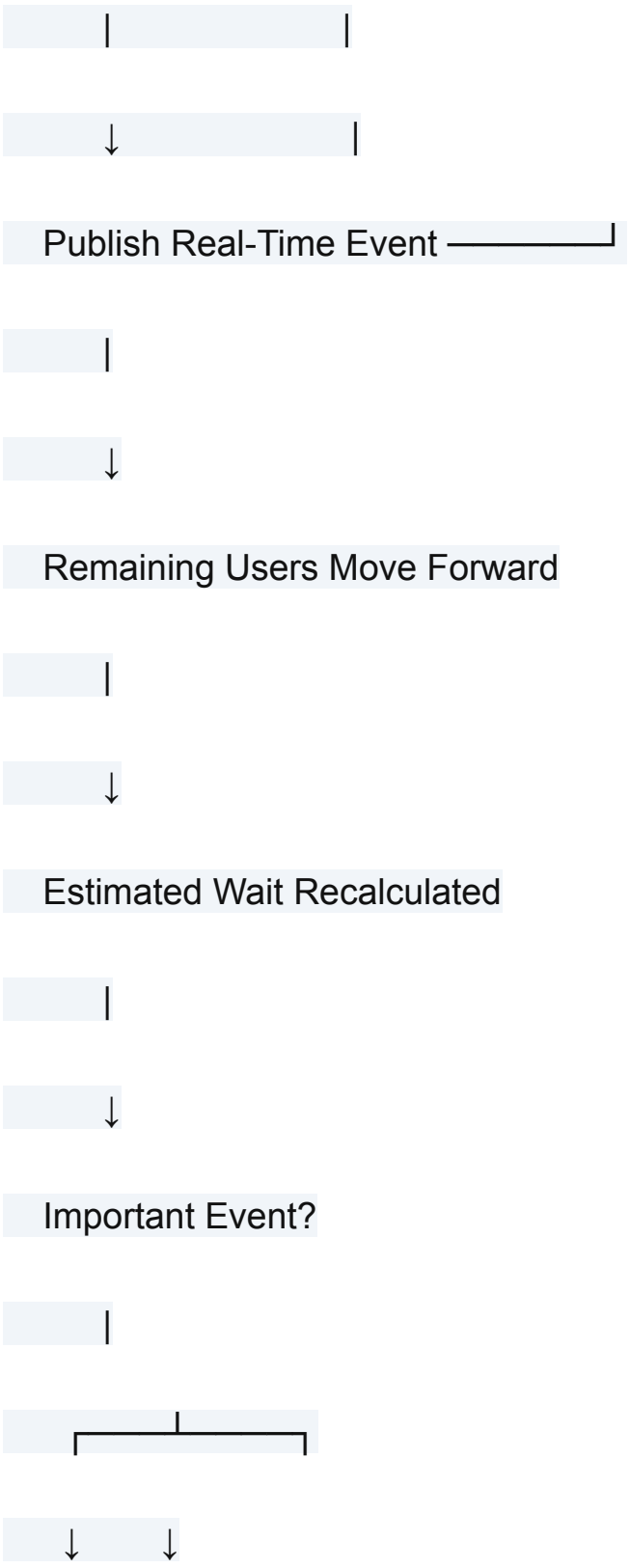
The complete system flow is:

```
                ADMIN

                  |

              Create Queue

                  |
```


Generate QR
|
↓
QR DISPLAYED
|
↓
USER
|
Scan QR
|
Enter Details
|
↓
Next.js Frontend
|


↓
Node.js API
|
Validate Request
|
Check Queue
|
↓
PostgreSQL
|
Create Entry
|
Assign Position
|
↓


User Gets #N
Real-Time Queue
Operator
User
Click DONE
View Position
Node.js Backend
Update Database


Publish Real-Time Event
Remaining Users Move Forward
Estimated Wait Recalculated
Important Event?

WhatsApp SMS

---

71. Example Complete Queue Scenario

Suppose 50 people join.

Person 1 → #1

Person 2 → #2

Person 3 → #3

...

Person 50 → #50

Person 1 completes their work.

Operator clicks:

DONE

System:

Person 1 → COMPLETED

Active queue becomes:

Person 2 → #1

Person 3 → #2

Person 4 → #3

...

Person 50 → #49

All connected users receive the updated state.

If Person 4 was previously:

#3

and two people ahead complete:

#3 → #1

the user sees the change automatically.

If their turn becomes available:

Turn Available

the notification system can send:

WhatsApp

+

SMS

according to configuration.

---

72. Architecture Acceptance Criteria

The architecture is considered ready when:

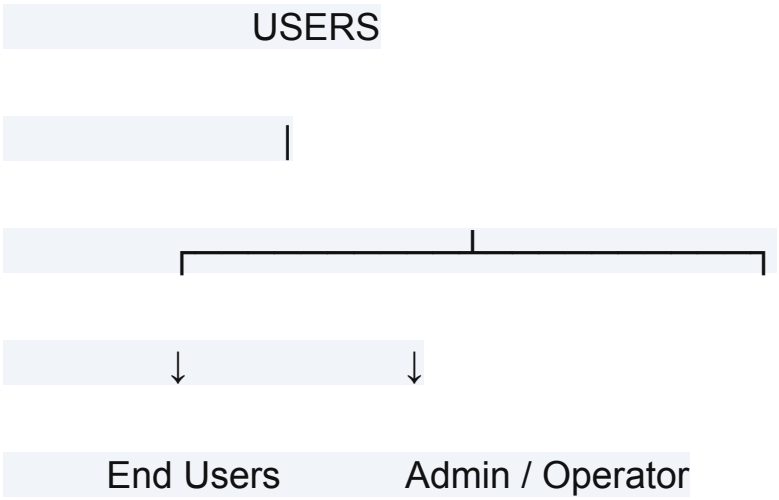
- Next.js frontend architecture defined.
- Node.js backend architecture defined.
- PostgreSQL integration defined.
- Multi-tenant architecture defined.
- Tenant isolation defined.
- Authentication defined.
- Authorization defined.
- QR generation flow defined.
- Queue joining flow defined.
- FCFS queue behavior defined.
- Queue position behavior defined.
- DONE flow defined.
- Admin reordering defined.
- User cancellation defined.
- Queue-clearing behavior defined.
- Estimated waiting-time architecture defined.
- Real-time architecture defined.
- Real-time recovery defined.
- WhatsApp notification architecture defined.
- SMS notification architecture defined.
- Environment configuration defined.
- API security defined.
- Validation defined.

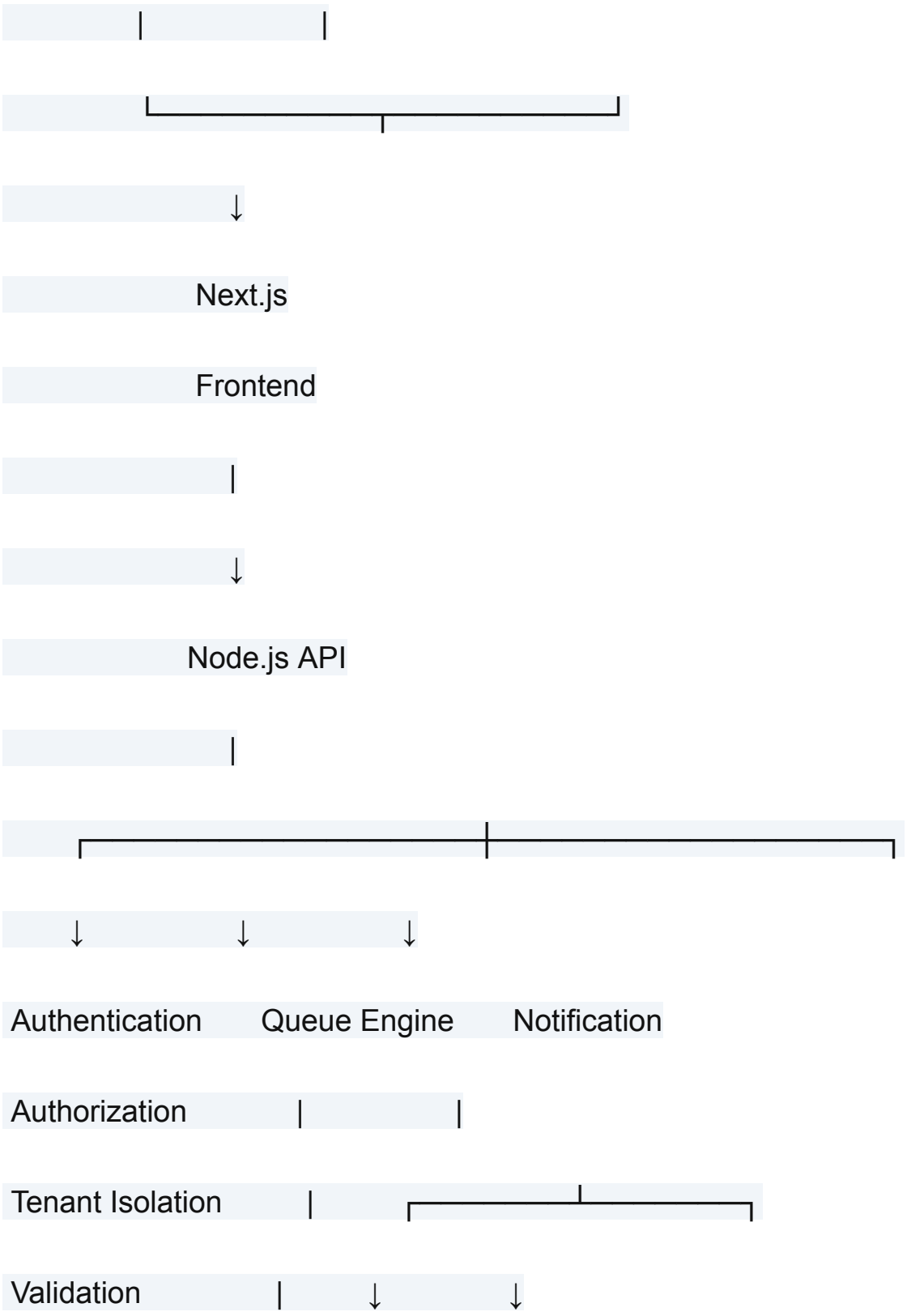
- Rate limiting defined.
- Database transactions defined.
- Concurrency requirements defined.
- Error handling defined.
- Responsive architecture defined.
- Shadcn UI integration defined.
- Playwright strategy defined.
- Selenium strategy defined.
- Husky workflow defined.
- Git branching defined.
- Monorepo approach evaluated.
- Logging approach defined.
- Monitoring approach considered.
- Deployment approach considered.

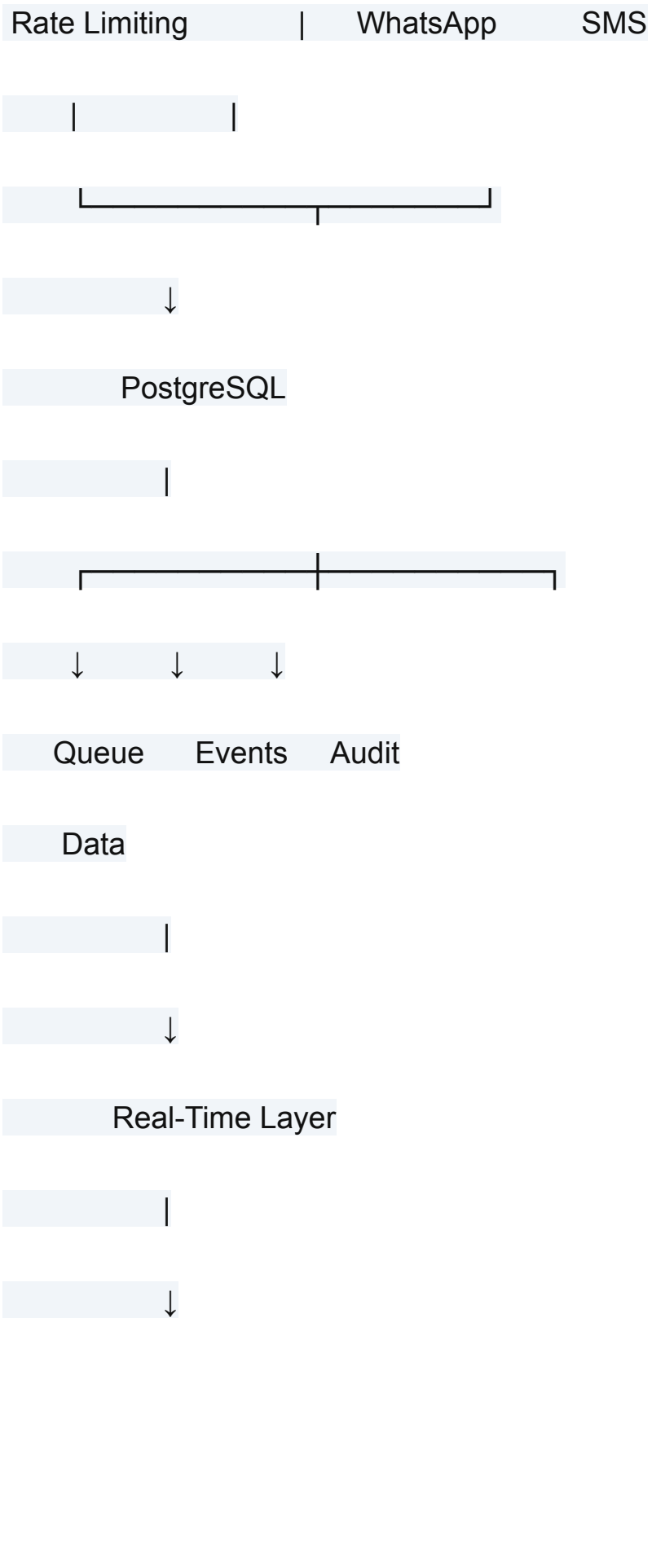
---

73. Final Architecture

The final logical architecture is:




Next.js
Frontend
Node.js API
Authentication
Queue Engine
Notification
Authorization
Tenant Isolation
Validation


Rate Limiting | WhatsApp SMS
PostgreSQL
Queue Events Audit
Data
Real-Time Layer

Connected Clients

---

74. Architecture Summary

The system is fundamentally a **multi-tenant queue management application**.

Its core architecture is:

Next.js

↓

Node.js

↓

PostgreSQL

with:

Multi-Tenant Isolation

+

Real-Time Queue Updates

+

WhatsApp

+

SMS

+

Authentication

+

Authorization

+

Validation

+

Rate Limiting

+

Automated Testing

The system keeps the queue logic centralized in the backend and stores authoritative state in PostgreSQL.

Users interact primarily through a responsive web application after

scanning an Admin-generated QR code.

The architecture deliberately avoids unnecessary industry-specific resource management and remains suitable for different queue-based use cases.

END OF ARCHITECTURE DOCUMENT